Lab 4

MPI Lattice Boltzmann

Parallel communication schemes for a D2Q9 CFD simulation

Author
Patron Antoine

Collaboration
Solo work

Exercises
1 – 6

M1 Info / MOSIG — Parallel Algorithms and Programming — Spring 2026

Scope of work

All six non-optional exercises were implemented and validated

1 1D blocking

MPI_Send / MPI_Recv

2 1D odd/even

Break the chain of dependencies

3 1D non-blocking

MPI_Isend / MPI_Irecv + Waitall

4 2D — manual copy

Cartesian topology + buffers

5 2D — MPI datatypes

MPI_Type_vector, stride-aware

6 2D non-blocking

Two batches to avoid corner dup.

Validation methodology

Three checks run for every exercise

1 check_comm — position

Exhaustive per-cell values. Verifies that the right data is stored at the right place in every ghost cell, across all patterns.

```
mpirun -np 4 ./check_comm \
  -e N -p position -s both
```

2 check_comm — modulo9

Compact readable pattern. Makes subtle off-by-one errors and wrong neighbor mappings visible in the terminal.

```
mpirun -np 4 ./check_comm \
  -e N -p modulo9 -s both
```

3 md5 checksum vs ex0

Compares the rendered frame at step 3 against the sequential reference. Guarantees bit-exact equivalence.

```
mpirun -np 4 ./lbm -e N
./display --gnuplot \
  output.raw 3 \
  | md5sum -c ref.md5
```

All three checks pass for exercises 1 through 6

Exercise 1 — 1D splitting, blocking comms

Baseline along the X axis with MPI_Send / MPI_Recv

- Split the global mesh along X only: nb_x = comm_size, nb_y = 1
- Each sub-domain has width total_width / nb_x + 2 ghost columns
- Borders handled by routing to MPI_PROC_NULL — no branching needed
- Matching tags + alternating send/recv order (Right then Left) prevents deadlocks in the MPI implementation

```
const int N = height * DIRECTIONS;
const int L = (rank_x > 0)      ? rank-1
                                : MPI_PROC_NULL;
const int R = (rank_x < nb_x-1) ? rank+1
                                : MPI_PROC_NULL;

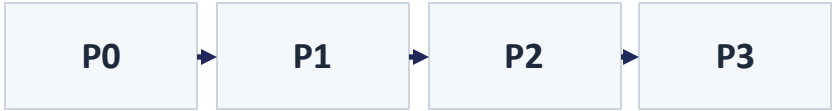
// Right, then recv from left
MPI_Send(cell(mesh, W-2, 0), N, MPI_DOUBLE,
         R, TAG_R, MPI_COMM_WORLD);
MPI_Recv(cell(mesh, 0,   0), N, MPI_DOUBLE,
         L, TAG_R, MPI_COMM_WORLD,
         MPI_STATUS_IGNORE);

// Left, then recv from right
MPI_Send(cell(mesh, 1,   0), N, ..., L, TAG_L, ...);
MPI_Recv(cell(mesh, W-1, 0), N, ..., R, TAG_L, ...);
```

Exercise 2 — 1D odd/even ordering

Breaking the serial dependency chain while staying blocking

Naive ordering



Serial chain: P0 → P1 → P2 → P3

Odd/even ordering



Parallel pairs — transfers happen simultaneously

```
const int is_even = (rank_x % 2 == 0);

if (is_even) {
  MPI_Recv(cell(mesh, 0,   0), N, MPI_DOUBLE,
           L, TAG_R, MPI_COMM_WORLD, ...);
  MPI_Send(cell(mesh, W-2, 0), N, MPI_DOUBLE,
           R, TAG_R, MPI_COMM_WORLD);
} else {
  MPI_Send(cell(mesh, W-2, 0), ...);
  MPI_Recv(cell(mesh, 0,   0), ...);
}
// symmetric block for the left direction
```

Init is reused as-is from ex1 — only the exchange order changes.

Exercise 3 — 1D non-blocking

MPI_Isend / MPI_Irecv with a single MPI_Waitall

- All receives posted first, then all sends — lets the MPI runtime overlap and match eagerly
- One MPI_Waitall at the end keeps the function synchronous from the caller's standpoint
- 4 requests stored in comm->requests (2 recv + 2 send)
- Same init as ex1 — only the exchange strategy differs

```
// Receives first (matched ASAP by the runtime)
MPI_Irecv(cell(mesh, 0,   0), N, MPI_DOUBLE,
          L, TAG_R, MPI_COMM_WORLD, &req[0]);
MPI_Irecv(cell(mesh, W-1, 0), N, MPI_DOUBLE,
          R, TAG_L, MPI_COMM_WORLD, &req[1]);

// Then sends
MPI_Isend(cell(mesh, W-2, 0), N, MPI_DOUBLE,
          R, TAG_R, MPI_COMM_WORLD, &req[2]);
MPI_Isend(cell(mesh, 1,   0), N, MPI_DOUBLE,
          L, TAG_L, MPI_COMM_WORLD, &req[3]);

MPI_Waitall(4, req, MPI_STATUSES_IGNORE);
```

Exercise 4 — 2D splitting with manual copy

Cartesian topology, 8 neighbors, buffered non-contiguous rows

- MPI_Dims_create picks a balanced 2D process grid automatically
- MPI_Cart_create (non-periodic) + MPI_Cart_coords give each rank its (rank_x, rank_y)
- Column-major layout → left/right are contiguous (H×9 doubles, direct send)
- Top/bottom rows are scattered: pack into buffer_send_up / _down, send, unpack into ghost rows
- All 4 buffers allocated once in init, freed in release — no allocation in the hot loop
- Corners are carried implicitly through the horizontal send (ghost columns already include corners)

```
// Pack non-contiguous top/bottom rows, then send
for (int x = 0; x < W; ++x)
  memcpy(&buffer_send_up[x * DIRECTIONS],
         cell(mesh, x, 1), DIRECTIONS * sizeof(double));

MPI_Sendrecv(buffer_send_up,   row_count, MPI_DOUBLE, up,   2,
             buffer_recv_down, row_count, MPI_DOUBLE, down, 2,
             comm->communicator, MPI_STATUS_IGNORE);
```

Exercise 5 — 2D with MPI datatypes

Replacing manual copies by a single MPI_Type_vector

- Create the type once in init, commit it, store in comm->type, free in release
- The stride parameter matches the column-major layout: one column is height × DIRECTIONS doubles
- Net effect: the manual pack/unpack loops of ex4 disappear entirely
- Corners are exchanged separately as contiguous DIRECTIONS-doubles buffers (Sendrecv with MPI_DOUBLE)

```
// count = W cells; blocklength = 9 doubles per cell;
// stride = height*9 doubles — distance between two cells on the same row
MPI_Type_vector(comm->width, DIRECTIONS,
                comm->height * DIRECTIONS,
                MPI_DOUBLE, &comm->type);
MPI_Type_commit(&comm->type);

// Top/bottom exchange: one call, no temporary buffer
MPI_Sendrecv(cell(mesh, 0, 1),     1, comm->type, up,   2,
             cell(mesh, 0, H - 1), 1, comm->type, down, 2,
             comm->communicator, MPI_STATUS_IGNORE);
```

Exercise 6 — 2D non-blocking

Two-batch exchange to prevent duplicate corner writes

- Init and release inherit from ex5 (same MPI datatype)
- Batch 1 — cardinal directions (left, right, up, down): 4 Irecv then 4 Isend, one Waitall(8)
- Batch 2 — diagonals (4 corners, DIRECTIONS doubles each): another 4 Irecv + 4 Isend, Waitall(8)
- Splitting the batches prevents a corner cell from being overwritten twice as the subject warns
- 8 requests reused between batches since the first Waitall completes them all

```
// Batch 2 — diagonals. Tags are crossed so each pair matches:
// what I send to up_left arrives as coming from down_right.
MPI_Irecv(cell(mesh, 0,     0),     9, MPI_DOUBLE, up_left,    4, comm_, &req[0]);
MPI_Irecv(cell(mesh, W - 1, 0),     9, MPI_DOUBLE, up_right,   5, comm_, &req[1]);
MPI_Irecv(cell(mesh, 0,     H - 1), 9, MPI_DOUBLE, down_left,  6, comm_, &req[2]);
MPI_Irecv(cell(mesh, W - 1, H - 1), 9, MPI_DOUBLE, down_right, 7, comm_, &req[3]);

MPI_Isend(cell(mesh, 1,     1),     9, MPI_DOUBLE, up_left,    7, comm_, &req[4]);
// ... three more Isend (up_right: 6, down_left: 5, down_right: 4)
MPI_Waitall(8, req, MPI_STATUSES_IGNORE);
```

Cross-cutting design choices

Patterns reused across all six exercises

MPI_PROC_NULL for borders

Every neighbor lookup at the global edge returns MPI_PROC_NULL, turning Send / Recv into no-ops. No special-case branching inside the communication loop.

Sendrecv where possible (ex4–5)

A single MPI_Sendrecv call handles one direction pair in a deadlock-free way. The implementation can internally match sends and receives across all ranks.

Receives posted before sends (ex3, ex6)

Posting Irecv first gives the MPI runtime information it can use to place incoming messages directly, avoiding intermediate buffering.

lbm_neighbor_rank helper

A small static helper centralises the out-of-bounds → MPI_PROC_NULL logic and the MPI_Cart_rank call. Cuts repetition across ex4/5/6.